

Part 02

1. The main difference is that a class is a reference type, while a struct is a value type.
 - The main difference is that **a class is a reference type**, while **a struct is a value type**.

Class	Struct
Reference type	Value type
Variables hold a reference to an object	Variables contain the value
Can inherit from another class	Cannot inherit from a class
Can be used as a base class	Cannot be a base class
Can be <code>null</code>	Normally cannot be <code>null</code> *
Supports inheritance	No class inheritance
Usually used for larger/complex objects	Usually used for small data values

2. If inheritance is relation between classes clarify other relations between classes

Yes. In OOP, **inheritance is only one type of relationship between classes**. There are several important relationships you should know:

1. Inheritance — "is-a"

A class is a specialized version of another class.

```
</> C#  
  
class Animal { }  
  
class Dog : Animal { }
```

A Dog **is an** Animal.

2. Association — "uses/knowns-a"

One class is connected to or uses another class, but neither necessarily owns the other.

```
</> C#  
  
class Teacher  
{  
    public void Teach(Student student)  
    {  
        // ...  
    }  
}
```

A Teacher **uses/knowns a** Student.

The two objects can exist independently.

3. Aggregation — "has-a" (weak ownership)

One class contains or groups other objects, but the contained objects can exist independently.

```
</> C#  
  
class Department  
{  
    public List<Teacher> Teachers { get; set; }  
}
```

A `Department` **has** `Teachers`.

If the department is deleted, the teachers can still exist.

4. Composition — "has-a" (strong ownership)

One class owns another object, and the contained object's lifetime is strongly tied to the owner.

```
</> C#  
  
class Car  
{  
    private Engine engine = new Engine();  
}  
  
class Engine  
{  
}
```

A `Car` **has an** `Engine`.

Conceptually, if the `Car` is destroyed, its `Engine` belongs to that car and isn't independently managed in the same way as an aggregated object.

5. Dependency — "depends-on"

One class temporarily depends on another class to perform a task.

```
</> C#  
  
class Report  
{  
    public void Print(Printer printer)  
    {  
        printer.Print();  
    }  
}
```

`Report` **depends on** `Printer` to perform printing.